Национальный исследовательский университет «Высшая школа экономики»

Факультет компьютерных наук

Департамент программной инженерии

Электронный аукцион

Отчёт по индивидуальному заданию № 2

по дисциплине «Архитектура компьютера и операционные системы»

Разработка многопоточной имитационной программы

Вариант 37

Выполнил: Рыжов Степан Игоревич

Группа: БПИ252

Москва 2026

Условие задачи и правила модели

Требуется разработать консольное приложение на C для Linux, моделирующее электронный аукцион. Несколько лотов продаются последовательно или параллельно. Каждый лот имеет начальную цену, минимальный шаг и срок закрытия. Участники располагают ограниченными бюджетами, используют выбранные стратегии и готовят ставки с заданной задержкой. Аукционист, участники и лоты являются самостоятельными сущностями модели.

Сценарий торгов

1. Аукционист открывает ожидающие лоты по номерам, соблюдая ограничение числа одновременно открытых лотов.

2. Участник выбирает первый открытый лот, где он ещё не лидирует и не исключён после отказа. Одновременно он готовит одну заявку. Стратегия 0 повышает текущую цену на один шаг, стратегия 1 - на два шага.

3. Сумма фиксируется при подготовке. После задержки заявка передаётся аукционисту. К этому моменту цена или состояние лота могут измениться.

4. Аукционист проверяет срок, номер раунда, минимальный шаг и доступные деньги. Первая ставка тоже должна превышать начальную цену как минимум на шаг.

5. При принятии ставки резервируется её полная сумма. В режиме 0 резерв прежнего лидера освобождается сразу; в режиме 1 он удерживается до закрытия раунда. Для участника в одном лоте хранится только резерв его последней принятой ставки.

6. Принятая ставка продлевает срок до «текущее время + интервал продления», если это позже прежнего срока. Нулевой интервал отключает продление.

7. После закрытия аукционист определяет победителя, освобождает все резервы этого лота и списывает цену покупки. При разрешённом отказе списания нет: лот открывается заново с начальной ценой и полной длительностью, отказавшийся исключается из этого лота.

8. Лот без ставок остаётся непроданным. Работа завершается после закрытия всех лотов и расчётов либо по сигналу пользователя.

Условия корректности

Доступная сумма для ставки: бюджет минус все резервы плюс собственный резерв в этом лоте. Новая ставка заменяет прежний резерв, а не прибавляется к нему целиком. Деньги другого лота использовать нельзя. Резервы не превышают бюджет, бюджет после оплаты неотрицателен.

У лота один текущий лидер и не более одного окончательного победителя. Ставка принимается только до срока закрытия и в текущем раунде. Каждый отказ исключает ещё одного участника, поэтому число повторных открытий конечно.

Потоки и их взаимодействие

Программа использует pthread. Создаются отдельные потоки для всех участников, всех лотов и один поток аукциониста. Основной поток увеличивает счётчик тактов, следит за прерыванием и ожидает завершения остальных. Всего потоков N + M + 2, где N - число участников, M - число лотов. Во встроенном примере их 8; при максимальных параметрах - 66.

Сущность	Представление и поведение
Участник	Participant и participant_thread: выбирает лот, готовит ставку, после задержки ставит флаг ready и уведомляет аукциониста.
Лот	Lot и lot_thread: проверяет срок закрытия, учитывает продления и повторные открытия, передаёт запрос close_requested.
Аукционист	Auctioneer и auctioneer_thread: принимает и отклоняет заявки, закрывает лоты, выполняет расчёты и открывает ожидающие лоты.
Ставка	Bid: данные заявки, время готовности, раунд и флаг передачи. Отдельный поток для данных заявки не требуется.

Мьютекс и семафор

Один pthread_mutex_t защищает общие данные и вывод. Проверка ставки и изменение цены, лидера, резервов и бюджетов выполняются под этим мьютексом. Две заявки не могут одновременно потратить одни деньги. Сообщения разных потоков не смешиваются. Во время паузы мьютекс не удерживается.

Для уведомлений используется один POSIX-семафор. Участник вызывает sem_post после готовности ставки, поток лота - после истечения срока. Аукционист ожидает через sem_wait и обрабатывает накопленные флаги: сначала запросы закрытия, затем готовые ставки, затем открытие ожидающих лотов. Семафор хранит уведомления, а флаги показывают конкретную работу. До обработки участник свою заявку не меняет.

Семафор создаётся через sem_open с именем, содержащим PID. Сразу вызывается sem_unlink; после sem_close объект освобождается. Этот способ поддерживается в Linux и macOS.

Время и асинхронность

Время модели - обычный счётчик: 0, 1, 2 и так далее. Основной поток делает паузу и увеличивает счётчик на единицу под мьютексом. Все сроки и задержки задаются в тактах. Замеры реального времени не выполняются. Участники и лоты независимо читают этот счётчик с паузой 1 мс. Основной поток не задаёт порядок их действий. Пока аукционист обрабатывает события под мьютексом, счётчик не меняется. Даже если запрос закрытия ещё не поступил, ставка при времени, равном сроку или превышающем его, отклоняется.

Планировщик влияет на порядок передачи заявок, поэтому итоги разных запусков могут различаться. Это ожидаемое свойство многопоточной версии. Случайные числа не используются.

Изменения относительно первой части

Правила торгов из auction.c и числовой формат входа сохранены. Последовательный цикл заменён файлами и функциями из threads.c. В auction.h добавлены мьютекс, семафор и флаги готовности. Отсчёт времени оставлен обычным счётчиком тактов, как в первой части. В main.c сохранены параметры и сигналы, в Makefile добавлена сборка с -pthread. В config.c добавлена проверка диапазонов, чтобы неверные параметры не приводили к зависанию. Проверки обновлены с учётом асинхронного выполнения.

Интерфейс, ввод и завершение

Консоль показывает открытие лота, подготовку и поступление ставки, принятие или причину отклонения, смену лидера, резервирование и освобождение денег, продление, закрытие, победителя, оплату и итоговые бюджеты. Те же сообщения записываются в файл лога через dprintf и vdprintf.

Сборка и запуск

```
make
./auction
./auction -f data/default.txt -l result.log -d 20
./auction -f tests/data/reserves.txt -r 1 -d 20 -l hold.log
make test
```

Нужны компилятор C, make и pthread. Без -f используются встроенные данные, совпадающие с data/default.txt. Параметр -l задаёт лог, по умолчанию auction.log; файл перезаписывается. Имена входного файла и лога должны различаться. Параметр -h выводит справку.

Параметр -d задаёт паузу между тактами от 0 до 10000 мс, по умолчанию 200 мс. Значение 0 включает быстрый режим с паузой 1 мс. Параметр -r задаёт правило резервирования: 0 - только у лидера, 1 - сохранение резервов прежних лидеров до закрытия раунда.

Формат исходных данных

Файл содержит целые числа, разделённые пробелами или переводами строк:

```
2 1 1 2 1
100 0 1 0
150 1 2 1
20 10 5
```

Первая строка: число участников, число лотов, одновременно открытые лоты, интервал продления и правило отказа. Далее для каждого участника: бюджет, стратегия, задержка ставки, отказ. Далее для каждого лота: начальная цена, шаг, длительность.

Допустимы 1-32 участника и лота; число одновременно открытых лотов от 1 до общего числа лотов. Бюджет и начальная цена - от 0 до 1000000, шаг - от 1 до 1000000. Длительность и задержка - от 1 до 1000 тактов, продление - от 0 до 1000. Стратегия, правило отказа и признак отказа равны 0 или 1. Проверяются диапазоны и полнота данных. Размер файла меньше 16383 байт. Для чтения используются open, read, close и strtol.

Завершение

Обычное завершение наступает после закрытия всех лотов. При Ctrl+C или SIGTERM отменяются незавершённые лоты и освобождаются их резервы. Оплаченные покупки сохраняются.

Рабочие потоки наследуют заблокированные SIGINT и SIGTERM; сигналы принимает основной поток. Обработчик только сохраняет номер сигнала. Основной поток выставляет флаг остановки под мьютексом и будит аукциониста через sem_post. Все потоки завершаются и присоединяются через pthread_join. После этого выводятся итоги, закрывается семафор, уничтожается мьютекс и закрывается лог. При ошибке создания потока уже созданные тоже останавливаются.

Коды выхода: 0 - обычное завершение; 1 - ошибка; 130 - SIGINT; 143 - SIGTERM.

Результаты проверки

Сборка выполнена без предупреждений с -std=c11 -Wall -Wextra -Wpedantic -pthread. Проверки make test пройдены в macOS arm64 с Apple Clang 14.0.3 и в Linux x86_64 в локальном контейнере с GCC 9.4.0.

Скрипт tests/check.sh запускает встроенный пример и файлы данных в двух режимах резервирования. Журнал каждого запуска сравнивается с его консольным выводом. Проверяются ожидаемые события, наличие итогов и освобождение резервов. В простом примере сверяются цена, единственная оплата, бюджет и выручка. Ожидание ограничено для обнаружения зависаний.

Проверка	Полученный результат
default, reserves, hold_refusal	Торги завершены в обоих режимах; бюджеты и резервы согласованы, в итогах резервы нулевые.
unsold, deadline	Лоты без доступных денег не проданы; заявка на границе срока не принята.
extension	Принятые ставки продлевают срок, после закрытия выполнена одна оплата.
all_refuse	Отказ единственного участника вызывает повторное открытие; лот остаётся непроданным.
Повторные быстрые запуски	Пять запусков в каждом режиме: проверяются завершение и совпадение консольного вывода с журналом при разном порядке событий.
many	32 участника и 32 лота, 66 потоков с учётом основного: штатное завершение, выручка 0.
SIGINT и SIGTERM	В обоих режимах незавершённые лоты отменены, резервы освобождены; коды 130 и 143.
paid_interrupt	После оплаты первого лота SIGTERM отменяет второй; оплата 15 и остаток 85 сохраняются.
Некорректный ввод	Неверные -r, -d и ноль одновременно открытых лотов отклонены с кодом 1.

Пример результата

При проверке data/default.txt с -r 0 и -d 20 получен следующий итог:

```
=== ИТОГИ: завершено, t=10 ===
Лот 1: продан, победитель 1, цена 60.
Лот 2: продан, победитель 1, цена 90.
Лот 3: не продан.
```

Участник 1 потратил 150, его остаток равен 10. Остатки участников 2 и 3 равны 220 и 120. Все резервы нулевые. Принято 6 ставок, отклонено 9, выполнено 2 продления, выручка 150. Это результат конкретного запуска, а не обязательный результат следующих запусков. Полный журнал находится в tests/default_r0.log.

Ограничения

Используется один общий мьютекс и короткие паузы, число участников и лотов ограничено 32. Сроки задаются в тактах модели. На коротких торгах результат сильнее зависит от планировщика; для наблюдения удобнее пауза 200 мс. Предусмотрено конечное число лотов.